

DEEP REINFORCEMENT LEARNING · COMPANION READER

Multi-Armed Bandits

The simplest RL setting, where exploration and exploitation collide: estimate each action's value, then balance trying new things against using what you know.

Session 2–3 (Lectures 2–3) · Read alongside the lecture slides · Sutton & Barto, Ch. 2

How to use this companion. The colour boxes guide you: **blue** intuition, **amber** everyday picture, **green** worked example, **red** watch out, **purple** key takeaway. Read it next to the slides, in order. Every slide example is worked in full.

1 What this session covers

The **multi-armed bandit** is RL stripped to one decision, repeated. There is no changing state — just k actions, each paying a random reward. It is the cleanest place to study the central tension of RL: **exploration vs exploitation**. We define the problem and the **value** of an action, estimate values by **sample averaging**, and select actions **greedily** and with ϵ -**greedy**. We make the estimate efficient with an **incremental update**, handle **non-stationary** problems with a constant step size, encourage exploration with **optimistic initial values** and **upper-confidence-bound (UCB)** selection, and finally learn a policy directly with **gradient bandits** and the **softmax**.

2 The k -armed bandit problem

You face, repeatedly, a choice among k actions. After each choice you receive a numerical reward drawn from a fixed (stationary) probability distribution that depends only on the action you chose. The objective: maximise the expected total reward over some time period.

The intuition

Imagine k slot machines (“one-armed bandits”). Each has its own hidden average payout. You do not know the averages. You must learn which arms pay best by pulling them — while not wasting too many pulls on bad ones. That is the whole game.

★ Everyday picture

Choosing a lunch spot near the office. Each restaurant is an “arm” with an unknown average enjoyment. You could always return to your current favourite (exploit) or try a new place (explore). Over a year of lunches you want the highest total enjoyment.

2.1 The value of an action

The **value** of an action is its expected (mean) reward. We write:

- A_t — the action selected at time step t .

- $q_*(a)$ — the *true* value of action a : $q_*(a) = \mathbb{E}[R_t \mid A_t = a]$.
- $Q_t(a)$ — our *estimate* of that value at time t .

✗ Watch out

If you knew every $q_*(a)$ exactly, the bandit problem would be trivial — always pull the highest. The entire difficulty is that you only have *estimates* $Q_t(a)$, and improving them requires spending pulls (exploring) that you could have spent earning reward (exploiting).

✓ Key takeaway

A k -armed bandit is a single repeated choice among k actions with unknown mean rewards $q_*(a)$. The goal is to estimate these values, $Q_t(a)$, and use them to maximise total reward.

3 Estimating values: the sample average

How do we estimate an action's value? Average the rewards it has actually paid. The **sample-average** estimate of action a after it has been chosen some number of times is:

$$Q_t(a) = \frac{\text{sum of rewards when } a \text{ taken before } t}{\text{number of times } a \text{ taken before } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}}. \quad (1)$$

Here $\mathbf{1}_{A_i=a}$ is 1 if action a was taken at step i and 0 otherwise. By the law of large numbers, as the count grows, $Q_t(a) \rightarrow q_*(a)$.

★ Everyday picture

You have eaten at a restaurant four times and rated the meals 7, 9, 8, 8. Your estimate of its value is the average, $(7 + 9 + 8 + 8)/4 = 8$. Each new visit refines the estimate. The sample average is just “the running mean of what you got.”

✓ Key takeaway

The sample-average method estimates an action's value as the mean of the rewards it has paid so far; with enough samples it converges to the true value $q_*(a)$.

4 Selecting actions: greedy and ε -greedy

Given estimates, how do we choose? The **greedy** action is the one with the highest current estimate:

$$A_t = \arg \max_a Q_t(a). \quad (2)$$

Pure greedy always exploits — it never tries an action that currently looks worse, even though that action's *true* value might be higher (the estimate could just be unlucky so far). The fix is **ε -greedy**: behave greedily most of the time, but with small probability ε pick a *random* action (uniformly over all actions), independent of the estimates.

🔍 The intuition

ε is a small “curiosity tax.” Most pulls go to the best-known arm; a slice ε goes to random arms, just to keep checking whether something better is hiding behind a noisy estimate.

In the limit, every action is sampled infinitely often, so every $Q_t(a) \rightarrow q_*(a)$.

 Worked example: ε -greedy with two actions and $\varepsilon = 0.5$ (slide Ex-1)

With $k = 2$ actions and $\varepsilon = 0.5$, what is the probability the *greedy* action is selected? Split into the two ways it can happen.

$$\begin{aligned} P(\text{greedy}) &= P(\text{greedy} \mid \text{greedy step}) P(\text{greedy step}) \\ &\quad + P(\text{greedy} \mid \text{random step}) P(\text{random step}) \end{aligned} \quad (3)$$

$$= (1)(1 - \varepsilon) + \left(\frac{1}{2}\right)(\varepsilon) = (1)(0.5) + (0.5)(0.5). \quad (4)$$

Step 1. Greedy step. With probability $1 - \varepsilon = 0.5$ we deliberately pick greedy: contributes $1 \times 0.5 = 0.5$.

Step 2. Random step. With probability $\varepsilon = 0.5$ we pick uniformly; the greedy action is one of two, so probability 0.5: contributes $0.5 \times 0.5 = 0.25$.

Step 3. Total. $0.5 + 0.25 = \mathbf{0.75}$. So even at a heavy $\varepsilon = 0.5$, the greedy action is still chosen 75% of the time.

 Worked example: which steps were forced explorations? (slide Ex-2)

A 4-armed bandit with sample averages, $Q_1(a) = 0$ for all a . The sequence is $A_1=1, R_1=1$; $A_2=2, R_2=1$; $A_3=2, R_3=2$; $A_4=2, R_4=2$; $A_5=3, R_5=0$. Track the estimates and ask when a non-greedy action was taken.

Step 1. $t = 1$. All estimates 0 (a tie); any action is “greedy.” Could be random, not forced.

Step 2. After $R_1=1$: $Q(1) = 1$, others 0. **$t = 2$:** greedy is action 1, but we chose 2 — this is **definitely** an exploration.

Step 3. After $R_2=1$: $Q(2) = 1 = Q(1)$, tie. **$t = 3$:** action 2 is (tied) greedy — *possibly* greedy, not forced.

Step 4. After $R_3=2$: $Q(2) = \frac{1+2}{2} = 1.5$, the max. **$t = 4$:** action 2 is greedy — *possibly* greedy.

Step 5. After $R_4=2$: $Q(2) = \frac{1+2+2}{3} \approx 1.67$, still the max. **$t = 5$:** we chose action 3, not the greedy 2 — **definitely** an exploration.

Step 6. Answer. Definitely exploring at $t = 2$ **and** $t = 5$; possibly at $t = 1, 3, 4$.

 Watch out

A higher ε finds the best action faster but then keeps wasting pulls on random (often inferior) actions forever, lowering the long-run average. A lower ε exploits more but is slow to discover the best arm. The right ε depends on how noisy the rewards are.

ML/AI connection. ε -greedy is the simplest exploration strategy and appears throughout deep RL (e.g. in DQN, with ε annealed over training). Easy to implement, easy to tune, and surprisingly effective.

 Key takeaway

Greedy selection always exploits the best estimate; ε -greedy adds a small random exploration so every action keeps being sampled, guaranteeing the estimates converge — at the

cost of some wasted pulls.

5 The 10-armed testbed

To compare methods fairly, the book uses the **10-armed testbed**: 2000 randomly generated 10-armed bandit problems. For each, the true values $q_*(a)$ are drawn from a normal distribution (mean 0, variance 1); when an action is taken, the actual reward is drawn from a normal with mean $q_*(a)$ and variance 1. One *run* applies a method for 1000 steps; averaging over 2000 runs gives a method's typical behaviour.

The 10-armed testbed: rewards spread around each arm's true value

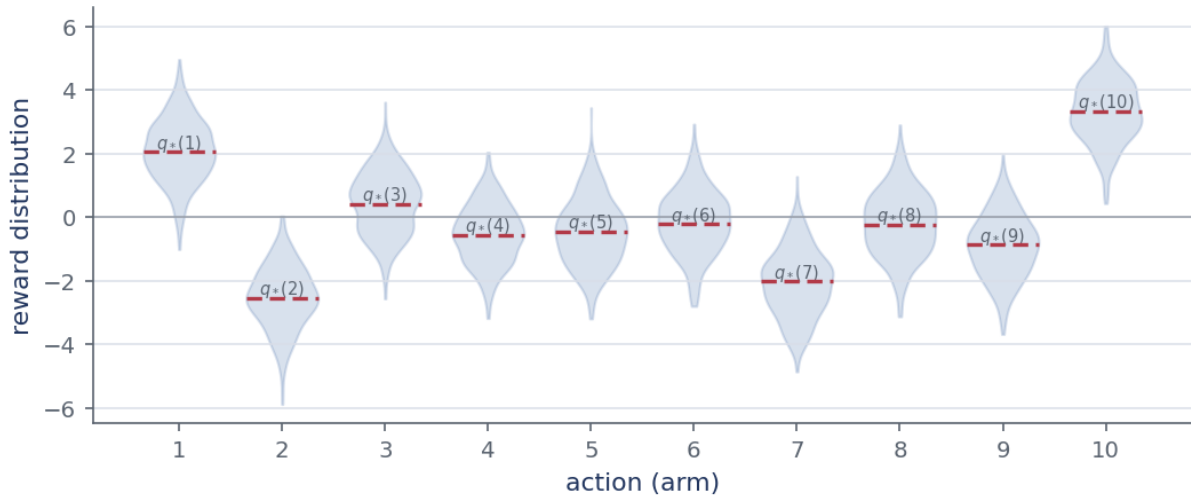


Figure 1: The 10-armed testbed. Each arm's reward is spread around its true value $q_*(a)$ (the dashed marks). The arms overlap, so a single sample can mislead — which is exactly why exploration is needed to tell them apart.

✗ Watch out

Comparing RL methods on a *single* run is meaningless — noise dominates. Always average over many independent runs (here 2000) before believing one method beats another.

✓ Key takeaway

The 10-armed testbed averages over 2000 noisy problems to compare bandit methods fairly — the standard way to evaluate exploration strategies.

6 The incremental update

Recomputing a sample average from scratch each step wastes memory and time. We can update it incrementally. Let Q_n be the estimate after $n - 1$ rewards and R_n the n th reward. The new average is:

$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i = \frac{1}{n} \left(R_n + \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} (R_n + (n-1)Q_n) = Q_n + \frac{1}{n} (R_n - Q_n). \end{aligned} \quad (5)$$

This is the familiar shape: $\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize}[\text{Target} - \text{OldEstimate}]$. Here the step size is $\frac{1}{n}$, which *shrinks* as more rewards arrive. We write the step size as α (constant) or $\alpha_t(a)$ (varying).

Worked example: three incremental updates

Start with $Q_1 = 0$ and receive rewards $R_1 = 10$, $R_2 = 0$, $R_3 = 8$ using step size $\frac{1}{n}$.

Step 1. $Q_2 = Q_1 + \frac{1}{1}(R_1 - Q_1) = 0 + 1 \cdot (10 - 0) = 10$.

Step 2. $Q_3 = Q_2 + \frac{1}{2}(R_2 - Q_2) = 10 + 0.5 \cdot (0 - 10) = 5$.

Step 3. $Q_4 = Q_3 + \frac{1}{3}(R_3 - Q_3) = 5 + \frac{1}{3}(8 - 5) = 5 + 1 = 6$.

Step 4. Check. The plain average of 10, 0, 8 is $18/3 = 6$. The incremental result matches exactly — no need to store the rewards.

Key takeaway

The incremental update $Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n)$ computes the running average in $O(1)$ memory — the template `new = old + step[target - old]` that runs through all of RL.

7 Non-stationary problems and recency weighting

Most real RL problems are **non-stationary**: the true action values change over time. A shrinking step size $\frac{1}{n}$ trusts old rewards as much as new ones — bad when the world has moved on. The fix: use a *constant* step size α , so recent rewards count more.

With a constant α , unrolling the update gives the **exponential recency-weighted average**:

$$Q_{n+1} = (1 - \alpha)^n Q_1 + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i. \quad (6)$$

The weight on the reward from i steps back is $\alpha(1 - \alpha)^{n-i}$: older rewards decay *geometrically*, and the weights sum to 1.

Worked example: recency weights with $\alpha = \frac{1}{2}$

Rewards in order $R_1=10, R_2=0, R_3=8, R_4=4$, with $Q_1 = 0$, $\alpha = \frac{1}{2}$.

Step 1. $Q_2 = 0 + \frac{1}{2}(10 - 0) = 5$.

Step 2. $Q_3 = 5 + \frac{1}{2}(0 - 5) = 2.5$.

Step 3. $Q_4 = 2.5 + \frac{1}{2}(8 - 2.5) = 5.25$.

Step 4. $Q_5 = 5.25 + \frac{1}{2}(4 - 5.25) = 4.625$.

Step 5. Weights view. The final weights on R_1, R_2, R_3, R_4 are 0.0625, 0.125, 0.25, 0.5 (each older reward halved). Cross-check: $0.0625(10) + 0.125(0) + 0.25(8) + 0.5(4) = 0.625 + 0 + 2 + 2 = 4.625$ — the same answer. The most recent reward (R_4) carries half the weight.

Watch out

With a constant α , the bias from the initial estimate Q_1 never fully disappears (the $(1 - \alpha)^n Q_1$ term only decays). With the sample-average $\frac{1}{n}$, the bias vanishes once each action is tried once. Constant α trades this permanent (but shrinking) bias for the ability

to track change.

✓ Key takeaway

For non-stationary problems use a constant step size α : it weights recent rewards exponentially more than old ones, letting the estimate track a moving target.

8 Optimistic initial values

Setting the initial estimates high is a simple trick to *force* early exploration. On the 10-armed testbed, set every $Q_1(a) = +5$ instead of 0.

🔍 The intuition

Whichever action the agent tries first, the real reward (around 0) is far below the optimistic +5 estimate. “Disappointed,” the agent lowers that estimate and tries a different action — which also disappoints — and so on. The result: every action gets tried several times early, before the estimates settle. Exploration, for free, with no ε .

⚠ Watch out

Optimistic initial values only help on *stationary* problems. On non-stationary ones they fail: the exploration push is a one-time effect at the start, so when the true values later change, there is nothing left to drive fresh exploration. It is a neat trick, not a general method.

✓ Key takeaway

Optimistic initial values encourage early exploration by making every first try a disappointment — effective on stationary problems, useless once the values shift.

9 Upper-confidence-bound (UCB) selection

ε -greedy explores *indiscriminately* — a random non-greedy action is as likely as any other, with no preference for promising or uncertain ones. **UCB** explores smarter: it prefers actions that are either high-valued or under-explored. It selects:

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right], \quad (7)$$

where $N_t(a)$ is the number of times a has been selected, $\ln t$ is the natural log of the time step, and $c > 0$ controls the degree of exploration.

🔍 The intuition

The first term $Q_t(a)$ rewards *value*; the second term $c\sqrt{\ln t/N_t(a)}$ rewards *uncertainty*. Each time a is selected, $N_t(a)$ grows and its bonus shrinks (less uncertain). Each time *another* action is selected, t grows but $N_t(a)$ does not, so a ’s bonus slowly grows (it has been neglected). UCB systematically gives every action its due, then focuses on the best.

✗ Watch out

UCB often beats ε -greedy on bandits, but it is harder to extend to the full RL setting (large state spaces, function approximation, non-stationarity). It is excellent for bandits, awkward beyond them.

✓ Key takeaway

UCB selects $\arg \max_a [Q_t(a) + c\sqrt{\ln t / N_t(a)}]$ — adding an uncertainty bonus that favours under-trying actions, exploring smarter than the random ε -greedy.

10 Gradient bandits and the softmax

The methods so far estimate *values* and pick from them. **Gradient bandit** algorithms instead learn a *preference* $H_t(a)$ for each action — a number with no meaning except relative to other preferences — and turn preferences into action probabilities with the **softmax**:

$$\pi_t(a) = \Pr(A_t = a) = \frac{e^{H_t(a)}}{\sum_{b=1}^k e^{H_t(b)}}. \quad (8)$$

After taking action A_t and receiving reward R_t , the preferences are updated by **gradient ascent** toward higher reward (with $\bar{R}_t$ the average reward so far as a baseline):

$$H_{t+1}(A_t) = H_t(A_t) + \alpha (R_t - \bar{R}_t) (1 - \pi_t(A_t)), \quad (9)$$

$$H_{t+1}(a) = H_t(a) - \alpha (R_t - \bar{R}_t) \pi_t(a), \quad a \neq A_t. \quad (10)$$

🧠 The intuition

If the action you took did better than your recent average ($R_t > \bar{R}_t$), push its preference *up* and the others *down*; if it did worse, do the opposite. The softmax then makes preferred actions more probable. You learn a *policy* directly, never storing action values at all.

★ Everyday picture

The softmax is how you'd weight your lunch choices: not “always the single best,” but “more often the ones I've enjoyed more.” Raise your fondness for a place that delighted you, lower it for one that disappointed, and pick roughly in proportion.

ML/AI connection. This is the seed of **policy-gradient** methods — the entire “policy-based” half of deep RL (REINFORCE, actor-critic, PPO). The softmax-over-preferences and the “(R_t – baseline)” update reappear, scaled up to neural networks, later in the course.

✓ Key takeaway

Gradient bandits learn action preferences $H_t(a)$, convert them to probabilities with the softmax, and update by gradient ascent on reward — learning a policy directly, the foundation of policy-gradient deep RL.

11 What we learned

The bandit problem is: choose the action with the highest expected reward, when those rewards are unknown, you face the same choice repeatedly, and you only observe the outcome of the

action you took. The approaches split two ways. **Value-based** methods estimate $Q_t(a)$ and act on it: greedy (with optimistic initialisation), ε -greedy, and UCB. **Policy-based** methods learn a policy directly: gradient ascent with the softmax. Throughout, the same incremental template — $\text{new} = \text{old} + \text{step}[\text{target} - \text{old}]$ — does the learning, and the same tension — explore vs exploit — shapes every choice.

A final note from the slides: the bandit is a **non-associative** task (one situation, repeated). When the best action depends on a *context* or **state**, we have an **associative** task — and that is the full reinforcement learning problem, which the next session formalises as a Markov decision process.

✓ **Key takeaway**

Bandits teach value-based (greedy, ε -greedy, optimistic, UCB) and policy-based (gradient/softmax) action selection under the explore–exploit tension. Add *context* and the bandit becomes the full RL problem: the MDP.